

INSTRUCCIONES DE TRABAJO — EQUIPO MIDDLEEND

PHP + Laravel + WebSocket + Excel

Sistema de Votaciones — Qué debe construir el equipo, exactamente

Para: los 4 integrantes del equipo Middleend

Nivel de partida: Principiante / Junior

Meta: que en una semana el equipo tenga entregados los 16 componentes obligatorios de la Sección 4, funcionando de forma integrada con Backend y Frontend.

Cómo leer este documento: Este documento es una instrucción de trabajo, no un curso. La Sección 4 ("Instrucciones: qué debe construir el equipo, exactamente") es la parte obligatoria — el resto son apoyos (herramientas, repaso de conceptos, troubleshooting y plan de días) para poder cumplirla.

Tabla de Contenidos

- 1. Introducción y Propósito
- 2. Kit de Supervivencia: Herramientas y Descargas
- 3. Antes de Empezar: Lo Mínimo que Deben Repasar
- 4. Instrucciones: Qué Debe Construir el Equipo, Exactamente
- 5. Trampas Comunes y Troubleshooting
- 6. Plan de Acción: Día 1 y Semana 1
- 7. Glosario Express

1. Introducción y Propósito

1.1 ¿Dónde encaja el equipo Middleend en el sistema completo?

El sistema de votaciones se construye entre 3 equipos, como 3 piezas de un mismo rompecabezas:

Equipo	Tecnología	Responsabilidad
Backend	Laravel, base de datos, lógica electoral	El cerebro: las reglas, los modelos y todo lo que decide qué puede pasar.
Frontend	Blade, Tailwind, JavaScript	La cara visible: pantallas, formularios, experiencia de usuario.
Middleend (ustedes)	PHP + Laravel + WebSocket + Excel	El "traductor" y el "sistema de tuberías": conecta a Backend con Frontend, mueve datos entre ellos en tiempo real, y convierte información hacia y desde Excel.

Ustedes son el puente entre lo que pasa "en el momento" (alguien vota, ahora mismo) y lo que se ve en pantalla; y también el puente entre archivos externos (listas de estudiantes en Excel) y la base de datos del sistema.

1.2 ¿Por qué esta combinación de tecnologías es clave?

En un sistema de votaciones real hay tres necesidades que no cubre ni el Backend puro ni el Frontend puro:

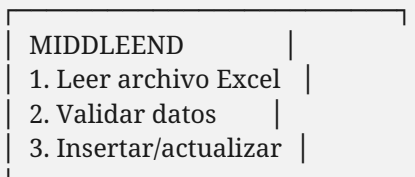
- Avisar en el instante: cuando alguien vota, los resultados en pantalla deben actualizarse solos, sin que nadie recargue la página. Esto no lo hace Laravel por sí solo — necesita un servidor aparte que "hable" en tiempo real.
- Traer datos desde Excel: las listas de estudiantes casi siempre llegan en una hoja de cálculo, no una por una a mano.
- Sacar datos hacia Excel: los resultados y reportes deben poder descargarse en un formato que cualquier persona pueda abrir e imprimir.

Laravel (el framework de PHP que usa todo el proyecto) es ideal como base porque ya trae herramientas para organizar código, y porque existen paquetes maduros y confiables para leer/escribir Excel (Laravel Excel, que verán en la Sección 4) y para todo lo demás construimos piezas propias (como el servidor de WebSocket).

Analogía simple: Piensen en su equipo como la sala de máquinas de un barco. Backend es el capitán que da las órdenes, Frontend es la cubierta que los pasajeros ven, y ustedes son la sala de máquinas: las tuberías, los cables y los motores que hacen que las órdenes del capitán realmente muevan el barco.

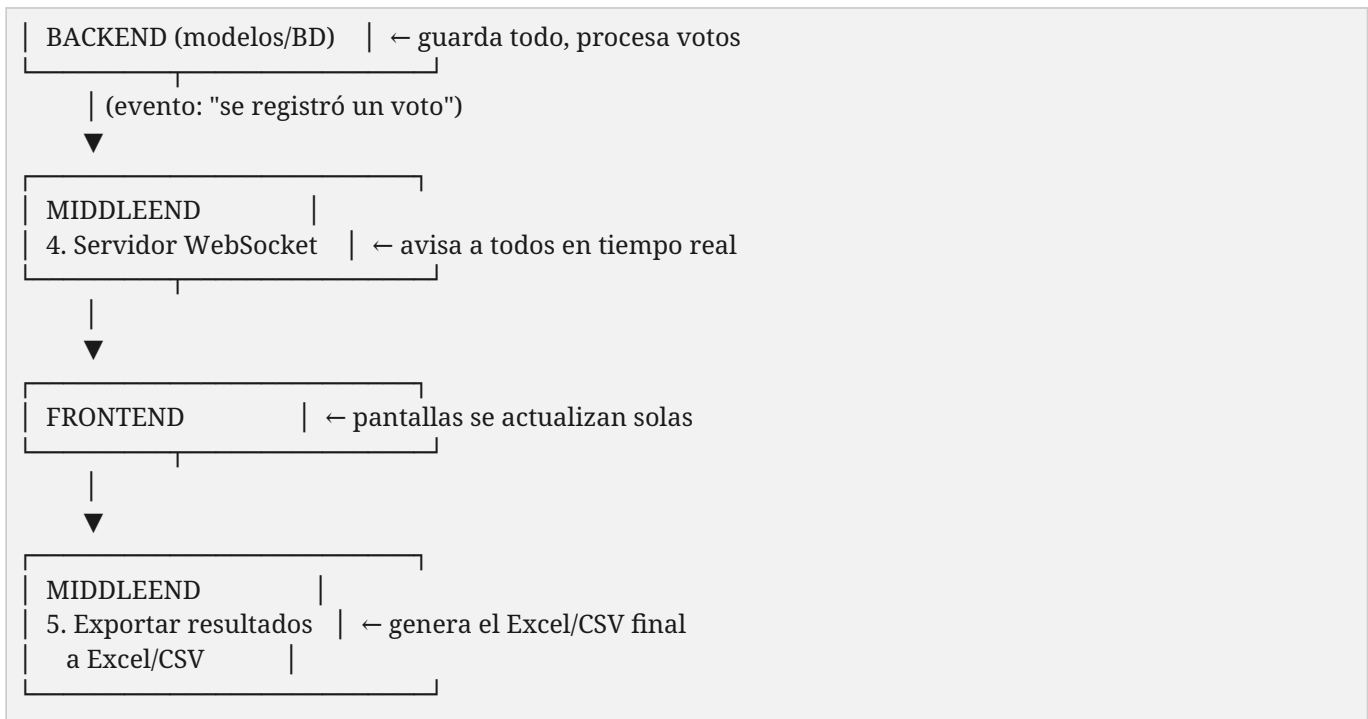
1.3 El flujo general que van a construir

[Excel: Lista de estudiantes]



↓ (datos limpios)





⚠ Tarea de coordinación importante: el "cómo" exactamente conversan Middleend, Backend y Frontend — ¿qué eventos existen?, ¿qué datos trae cada mensaje?, ¿qué columnas trae el Excel de estudiantes? — debe definirse en una reunión conjunta lo antes posible. Esta guía los prepara técnicamente, pero ese "contrato de integración" es una conversación que deben tener el Día 1 (ver Sección 6).

No se preocupen si hoy esto suena abstracto. Al terminar esta guía va a quedar muy claro.

2. Kit de Supervivencia: Herramientas y Descargas

Antes de escribir una sola línea de código, los 4 integrantes deben tener exactamente el mismo entorno instalado. Esto evita el clásico "en mi máquina sí funciona". Instalen todo en el mismo orden y, si pueden, la misma versión.

2.1 Checklist de instalación obligatoria

#	Herramienta	¿Para qué sirve?	Recomendación	Prioridad
1	PHP	El lenguaje en el que corre todo el proyecto	PHP 8.2 o superior	Obligatorio
2	Composer	Gestor de dependencias de PHP (como Maven, pero para PHP)	Última versión estable	Obligatorio
3	Node.js + npm	Compila los archivos de Frontend (Vite) que ustedes también van a necesitar correr	Versión LTS más reciente	Obligatorio
4	Editor de código	Donde van a escribir el código	VS Code (gratis, con extensión de PHP)	Obligatorio
5	Git	Control de versiones, para trabajar en equipo	Cualquier versión reciente + cuenta en GitHub/GitLab	Obligatorio

#	Herramienta	¿Para qué sirve?	Recomendación	Prioridad
6	Excel o LibreOffice Calc	Para crear y revisar archivos de prueba	Cualquiera de los dos sirve	Muy recomendado
7	Postman o Insomnia	Probar mensajes y comunicación con Backend/Frontend	Postman (gratis)	Opcional

Importante: El proyecto ya trae un archivo `composer.json` y `package.json`. Esto significa que no van a crear el proyecto desde cero: van a clonar el repositorio y "instalar" lo que esos archivos piden, con `composer install` y `npm install`.

2.2 Pasos de instalación (orden sugerido)

- 1. Instalar PHP 8.2+ y confirmar en una terminal con: `php -v`
- 2. Instalar Composer y confirmar con: `composer -V`
- 3. Instalar Node.js y confirmar con: `node -v` y `npm -v`
- 4. Instalar Git y clonar el repositorio compartido del proyecto.
- 5. Dentro de la carpeta del proyecto, correr: `composer install` (instala las librerías de PHP)
- 6. Luego correr: `npm install` (instala las librerías de Frontend/Vite)
- 7. Copiar el archivo `.env.example` a `.env` y ajustar los valores necesarios (ver Sección 4.6).
- 8. Correr: `php artisan key:generate`
- 9. Confirmar que los 4 integrantes puedan hacer `git clone`, `git pull` y `git push` sin errores.

Meta de esta sección: Al terminar, los 4 deben poder levantar el proyecto con `php artisan serve`, ver la pantalla de inicio en el navegador, y subir/bajar cambios con Git.

3. Antes de Empezar: Lo Mínimo que Deben Repasar

Esto NO es un curso — es la lista mínima de conceptos que cada integrante necesita para poder ejecutar las instrucciones de la Sección 4. Investíguenlo por su cuenta antes o durante el Día 1-2 (ver Sección 6), en sesiones cortas de 15-20 minutos por tema, y pasen directo a programar.

Si van a trabajar en...	Repasen antes
Helpers (NumeroALetras, WebSocketBroadcast)	Funciones y métodos estáticos en PHP; clases y namespaces básicos
AppServiceProvider / directiva Blade	Qué es un Service Provider en Laravel y cómo se registra una directiva Blade
StudentsImport	El paquete maatwebsite/excel: interfaces ToCollection, WithHeadingRow, WithValidation, WithLimit, SkipsOnFailure
websocket_server.php	Diferencia entre una petición HTTP normal y una conexión WebSocket; qué es un "handshake"; qué es hacer "broadcast" a varios clientes
Configuración (.env, composer.json, package.json, vite.config.js)	Qué es una dependencia y cómo se instala con Composer/npm; para qué sirve el archivo .env
Seeders y factories	El sistema de Seeders de Laravel (<code>php artisan db:seed</code>)
Validaciones compartidas	El sistema de validación de Laravel (<code>Validator::make()</code> , reglas como

Si van a trabajar en...	Repasen antes
	required, numeric)
Cifrado de votos	La fachada Crypt de Laravel (Crypt::encryptString() / Crypt::decryptString())
Exportación CSV	Cómo generar una respuesta de descarga en Laravel (response()->streamDownload())
Testing	Conceptos básicos de PHPUnit y cómo Laravel lo integra (php artisan test)

Recursos: Documentación oficial recomendada: laravel.com/docs (Service Providers, Blade, Validation), docs.laravel-excel.com (Import/Export), y MDN — "The WebSocket API" para entender el concepto de conexión en tiempo real.

4. Instrucciones: Qué Debe Construir el Equipo, Exactamente

Esta es la parte más importante del documento. No es una sugerencia ni un tutorial: es la lista de entregables que el equipo Middleend debe producir, según la documentación técnica del proyecto. Cada fila de la siguiente tabla es una tarea obligatoria.

4.0 Tabla de entregables obligatorios

#	Entregable	Ruta en el proyecto	Qué debe hacer exactamente
1	NumeroALetras.php	app/Helpers/	Método estático que convierte un número a su forma escrita en palabras (enteros, decimales hasta centavos, negativos, hasta millones).
2	WebSocketBroadcast.php	app/Helpers/	Cliente que envía mensajes al servidor WebSocket: send() genérico, votoRegistrado() (tipo 6), urnaCambioEstado() (tipo 4/5), aperturaCierre() (tipo 3).
3	Directiva Blade @num2letras	app/Providers/AppServiceProvider.php	Registrar la directiva para que cualquier vista pueda usar @num2letras(\$numero) e invocar NumeroALetras::convertir().
4	StudentsImport.php	app/Imports/	Leer un archivo Excel de estudiantes, validar identificación, nombre, apellidos y seccion, e insertar o actualizar por identificación. Límite: 2,000 filas por importación. Debe contar inserted, updated y errors.
5	websocket_server.php	raíz del proyecto	Servidor standalone en el puerto 8080 (protocolo RFC 6455): aceptar conexiones, hacer handshake, validar el token (WS_ACCESS_TOKEN o APP_KEY con hash_equals), reenviar (broadcast) mensajes tipo 3/4/5/6 a todos los clientes conectados, y manejar ping/pong y cierre de conexión.
6	iniciar_servidor.bat	raíz del proyecto	Abrir automáticamente dos procesos: el servidor WebSocket (websocket_server.php) y el servidor de Laravel (php artisan serve en 0.0.0.0:8000).
7	composer.json / package.json / vite.config.js	raíz del proyecto	Configuración de dependencias de PHP y de Frontend, y configuración del build con Vite, listas para que cualquier integrante clone y corra el proyecto.
8	.env / config/	raíz y config/	Variables de entorno (APP_KEY, conexión a la base SQLite, WS_ACCESS_TOKEN) y los 10 archivos de configuración estándar de Laravel.
9	Seeders y factories	database/seeders/	DatabaseSeeder con datos de demostración para poder probar el sistema sin depender de datos reales.
10	Validaciones compartidas	a definir con Backend	Reglas de validación reutilizables para que Backend y Frontend no dupliquen lógica de validación.
11	Cifrado/descifrado de votos	usado en VoteController	Cifrar cada voto con Crypt antes de guardarlo, y descifrarlo únicamente al momento de contar resultados.
12	Funciones de exportación (CSV)	a definir con Backend	Generar archivos .csv descargables con resultados/reportes del sistema.

#	Entregable	Ruta en el proyecto	Qué debe hacer exactamente
13	Integración entre módulos	transversal	Confirmar que los datos que entrega Backend combinen correctamente con lo que necesita mostrar Frontend.
14	Documentación de API	a definir	Dejar documentado cómo se usan los helpers, el WebSocket y las importaciones/exportaciones que el equipo construyó.
15	Testing unitario y de integración	tests/	Pruebas automatizadas (PHPUnit) para los helpers, el validador de importación y los flujos críticos.
16	Manejo de errores y logging	transversal	Registrar errores relevantes con el sistema de logging de Laravel (Log::error(), Log::info()) en vez de dejar que fallen en silencio.

Definición de Terminado: Regla práctica: si un entregable de esta tabla no existe, no compila, o no cumple lo que dice su columna "qué debe hacer exactamente", esa tarea NO está terminada — sin importar cuánto se haya investigado sobre el tema. El criterio de avance es código que funciona, no solo teoría leída.

⚠ El orden recomendado para construir estos entregables es: Helpers (1-3) → Configuración (7-8) → WebSocket (5-6) → Importación (4) → Cifrado y validaciones (10-11) → Exportación (12) → Testing y documentación (14-16). Esto coincide con la dependencia real entre piezas: no pueden probar el WebSocket sin la configuración lista, ni las exportaciones sin tener datos que exportar.

4.1 Detalle de cada entregable

A continuación se explica, uno por uno, cómo se construye cada pieza de la tabla anterior.

Entregable 1-3 — Helper: Convertir números en palabras (NumeroALetras)

Se crea como una clase de apoyo con un método estático, guardada en `app/Helpers/NumeroALetras.php`:

```
namespace App\Helpers;

class NumeroALetras
{
    public static function convertir(float $numero): string
    {
        // Lógica que transforma, por ejemplo,
        // 125 en "ciento veinticinco"
        // ...
        return $texto;
    }
}
```

Luego, en `AppServiceProvider`, se registra como una directiva especial de Blade para poder usarla directamente dentro de las pantallas:

```
Blade::directive('num2letras', function ($expresion) {
    return "<?php echo \App\Helpers\NumeroALetras::convertir($expresion); ?>";
});
```

Y en cualquier vista del sistema, Frontend puede escribir simplemente `@num2letras($total)` y el número aparece escrito en palabras.

Entregable 2 — Helper: Avisar en tiempo real (WebSocketBroadcast)

Este helper es un "mensajero": cuando pasa algo importante (un voto, un cambio de estado), se conecta un instante al servidor WebSocket, envía un mensaje y se desconecta. No es el servidor en sí — es quien le avisa al servidor que algo pasó.

```
namespace App\Helpers;

class WebSocketBroadcast
{
    public static function send(array $mensaje): void
    {
        // Abre una conexión breve al servidor WebSocket (127.0.0.1:8080),
        // envía el mensaje en JSON junto con un token de autenticación,
        // y cierra la conexión.
    }

    public static function votoRegistrado($datos): void
    {
        self::send(['tipo' => 6, 'datos' => $datos]);
    }

    public static function urnaCambioEstado($activada): void
    {
        self::send(['tipo' => $activada ? 4 : 5]);
    }

    public static function aperturaCierre($abierta): void
    {
        self::send(['tipo' => 3, 'abierta' => $abierta]);
    }
}
```

Contrato de datos: Cada "tipo" de mensaje es un número que representa un evento distinto (3 = apertura/cierre, 4 = urna activada, 5 = urna desactivada, 6 = voto registrado). Esto lo debe acordar el equipo con Backend, porque son ellos quienes llaman a estos métodos justo después de guardar algo importante en la base de datos.

Entregable 5 — El servidor WebSocket (websocket_server.php)

Aquí está el concepto clave: este NO es parte de Laravel. Es un programa PHP aparte, independiente, que se queda corriendo todo el tiempo escuchando un puerto (el 8080), esperando conexiones.

Lo que debe hacer, paso a paso:

- 1. Escuchar el puerto 8080 y aceptar nuevas conexiones.
- 2. Cuando alguien se conecta, hacer el "handshake" (el saludo inicial del protocolo WebSocket, siguiendo la norma RFC 6455).
- 3. Revisar el token de autenticación (comparando con hash_equals, que es una forma segura de comparar contraseñas/tokens sin dar pistas a quien intente adivinarlo).
- 4. Guardar en una lista todos los clientes que están conectados en ese momento (por ejemplo, todas las pantallas de resultados abiertas).
- 5. Cuando llega un mensaje de un cliente autorizado (como WebSocketBroadcast), reenviarlo ("broadcast") a todos los demás clientes conectados.
- 6. Responder correctamente a los "ping" (para saber que la conexión sigue viva) y manejar cuando alguien se desconecta ("close frame").

 *Este es el trabajo más complicado del equipo. No es tan largo en líneas de código, pero requiere entender bien el protocolo antes de programarlo. Denle tiempo extra en el cronograma y no lo dejen para el final.*

Para arrancarlo, se ejecuta con:

```
php websocket_server.php
```

Y para que no haya que abrir dos terminales cada vez, se acompaña de un script (iniciar_servidor.bat) que abre automáticamente dos ventanas: una para el servidor WebSocket y otra para el servidor de Laravel (php artisan serve).

Entregable 4 — Importación: leer estudiantes desde un Excel (StudentsImport)

Usando Laravel Excel, se crea una clase de importación que sabe leer el archivo fila por fila y decidir qué hacer con cada una:

```
namespace App\Imports;

use Maatwebsite\Excel\Concerns\ToCollection;
use Maatwebsite\Excel\Concerns\WithHeadingRow;
use Maatwebsite\Excel\Concerns\WithValidation;
use Maatwebsite\Excel\Concerns\WithLimit;
use Maatwebsite\Excel\Concerns\SkipsOnFailure;

class StudentsImport implements ToCollection, WithHeadingRow,
    WithValidation, WithLimit, SkipsOnFailure
{
    public $inserted = 0;
    public $updated = 0;
    public $errors = 0;

    public function collection($filas)
    {
        foreach ($filas as $fila) {
            // Busca por 'identificacion': si existe, actualiza;
            // si no existe, inserta un estudiante nuevo.
        }
    }

    public function rules(): array
    {
        return [
            'identificacion' => 'required',
            'nombre'        => 'required',
            'apellidos'      => 'required',
            'seccion'        => 'required',
        ];
    }

    public function limit(): int
    {
        return 2000; // máximo de filas por importación
    }
}
```

Nota: Igual que en la validación de un dato cualquiera: el objetivo no es que el proceso se caiga si una fila viene mal. El objetivo es detectar, reportar y seguir con las filas válidas — por eso se usa `SkipsOnFailure` en vez de dejar que un error tumbe toda la importación.

Entregable 12 — Exportación: generar reportes en CSV

Según la documentación del proyecto, esta pieza entrega "funciones de exportación (CSV)": tomar información del sistema (por ejemplo, resultados o listas) y convertirla en un archivo .csv descargable, que se puede abrir en Excel o en cualquier hoja de cálculo.

La idea general (a definir en el contrato de datos con Backend) es algo como:

```
// Dentro de un método que arma la respuesta de descarga:

$encabezados = [
    'Content-Type' => 'text/csv',
    'Content-Disposition' => 'attachment; filename="resultados.csv"',
];

return response()->streamDownload(function () use ($datos) {
    $salida = fopen('php://output', 'w');
    fputcsv($salida, ['Candidato', 'Total de votos']); // encabezado
    foreach ($datos as $fila) {
        fputcsv($salida, [$fila->nombre, $fila->total]);
    }
    fclose($salida);
}, 'resultados.csv', $encabezados);
```

Importante: Este es un ejemplo ilustrativo para entender el concepto de exportar a CSV en Laravel. El nombre exacto de la función, qué datos exporta y desde qué pantalla se llama debe acordarse con Backend, que es quien sabe qué reportes existen y qué información contienen.

Entregables 7-8 — Configuración del entorno (.env y config/)

El archivo .env guarda datos que cambian según la computadora donde corre el proyecto (por ejemplo, la conexión a la base de datos, o el token del WebSocket). Nunca se sube a Git tal cual — cada integrante tiene su propia copia local basada en .env.example.

Variable	¿Para qué sirve?
APP_KEY	Llave de cifrado general de Laravel (se genera con <code>php artisan key:generate</code>)
DB_CONNECTION / DB_DATABASE	Le dice a Laravel dónde está la base de datos (en este proyecto, SQLite)
WS_ACCESS_TOKEN	Token que autentica los mensajes que llegan al servidor WebSocket

Entregable 11 — Cifrado de votos

Los votos no se guardan "en claro": se cifran antes de guardarse, usando el sistema de cifrado que ya trae Laravel (Crypt), y se descifran únicamente en el momento de contar resultados.

```
use Illuminate\Support\Facades\Crypt;

// Al guardar:
$votoCifrado = Crypt::encryptString($valorDelVoto);
```

```
// Al contar resultados:
$valorOriginal = Crypt::decryptString($votoCifrado);
```

Nota: Esto protege que, aunque alguien viera directamente la base de datos, no pueda saber qué votó cada persona con solo mirar la tabla.

Entregable 10 — Validaciones compartidas

La documentación del proyecto lista "validaciones compartidas (reglas de validación)" como responsabilidad de Middleend. La idea es que, en vez de que Backend y Frontend escriban cada uno sus propias reglas para el mismo dato, el equipo centralice reglas reutilizables (por ejemplo, con el sistema de validación que ya trae Laravel, `Validator::make()`) para que ambos equipos las reutilicen.

Dependencia: Qué reglas exactas hacen falta (formato de cédula, campos obligatorios, etc.) depende de los modelos que define Backend — por eso esta tarea se hace después de tener claridad sobre esos modelos, y en coordinación directa con ese equipo.

Entregable 15 — Pruebas: cómo saber que su código realmente funciona

Laravel ya trae PHPUnit integrado. Cada pieza importante (el helper de números a letras, el validador de importación, el servidor WebSocket en su lógica interna) debería tener al menos una prueba básica.

```
public function test_convierte_numero_a_letras()
{
    $resultado = \App\Helpers\NumeroALetras::convertir(125);
    $this->assertEquals('ciento veinticinco', $resultado);
}
```

Las pruebas se corren con:

```
php artisan test
```

Por qué importa: Las pruebas son la forma de demostrar, con evidencia y no con promesas, que cada pieza hace lo que debe hacer. Esto es clave cuando Backend y Frontend empiecen a depender de su trabajo.

5. Trampas Comunes y Troubleshooting

Todo principiante se atasca en los mismos lugares. Aquí están, para que cuando les pase (y les va a pasar) no pierdan horas — vengan directo a esta tabla primero.

5.1 "Class not found" o error de namespace

Causa: el namespace del archivo no coincide con la carpeta donde está guardado, o falta ejecutar composer dump-autoload.

Solución: revisar que el namespace de la primera línea de la clase coincida exactamente con la ruta de carpetas (app/Helpers → namespace App\Helpers), y correr composer dump-autoload.

5.2 El servidor WebSocket no conecta o "se cae" apenas arranca

Causa más común: el puerto 8080 ya está siendo usado por otro programa, o falta la extensión sockets de PHP habilitada.

Solución: cerrar cualquier otro proceso que use el puerto 8080, y revisar en el archivo de configuración de PHP (php.ini) que extension=sockets esté activada.

5.3 Los mensajes del WebSocket no llegan al Frontend

Causa común: el token de autenticación (WS_ACCESS_TOKEN) no coincide entre lo que envía WebSocketBroadcast y lo que espera websocket_server.php, o el Frontend se está conectando a un puerto distinto.

Solución: confirmar que ambos lados usen exactamente el mismo valor de token desde el .env, y revisar en las herramientas de desarrollador del navegador (pestaña Network → WS) si la conexión realmente se abrió.

5.4 La importación de Excel "no hace nada" o no inserta datos

Causa común: los nombres de las columnas del Excel no coinciden con los nombres que espera WithHeadingRow (Laravel Excel convierte los encabezados a minúsculas y con guion bajo, por ejemplo "Número de Identificación" se vuelve numero_de_identificacion).

Solución: revisar exactamente cómo quedan los encabezados esperados, e imprimir/depurar la primera fila leída antes de asumir que el problema está en la lógica de inserción.

5.5 Error "file not found" o "could not open workbook"

Causa: el archivo Excel está siendo usado por dos procesos a la vez o la ruta no es la correcta.

Solución: cerrar el archivo si está abierto en Excel mientras el sistema lo procesa, y confirmar la ruta completa con la que se está intentando leer/guardar el archivo.

5.6 Cambios en .env que no se aplican

Causa: Laravel guarda una versión "cacheada" de la configuración.

Solución: correr php artisan config:clear después de cualquier cambio en el .env durante desarrollo.

5.7 Regla de oro para cuando algo simplemente "no tiene sentido"

- 1. Lean el mensaje de error completo, buscando la primera línea que mencione un archivo del propio proyecto (no una clase de Laravel o de una librería) — ahí está el lugar exacto del problema.
- 2. Copien el mensaje de error exacto y búsquenlo (la comunidad de Laravel es enorme, casi cualquier error ya fue resuelto por alguien más).
- 3. Regla de los 30 minutos: si llevan más de 30 minutos atascados en lo mismo, pregúntenle a un compañero del equipo antes de seguir solos. Quedarse "pegado" en silencio es lo único que de verdad atrasa al grupo.

6. Plan de Acción: Día 1 y Semana 1

6.1 División de roles (los 4 integrantes, desde el primer día)

Para que nadie se quede sin hacer nada y todos aprendan de todo, cada integrante tiene un rol principal — pero todos pasan por todas las partes del código en algún momento de la semana.

Integrante	Rol	Responsabilidad principal	Lo primero que domina
1	El Arquitecto	Entorno de todos, configuración (.env, composer.json, vite.config.js) y puente de comunicación con Backend y Frontend	Capas 1, 2 y 4 (fundamentos + Composer)
2	El Mensajero	Helpers (NumeroALetras, WebSocketBroadcast) y el servidor websocket_server.php	Capas 1, 2 y 6 (fundamentos + tiempo real)
3	El Guardián de datos	Importación de estudiantes (StudentsImport), validaciones compartidas y cifrado de votos	Capas 1, 3 y 5 (errores + Laravel Excel)
4	El Exportador	Exportaciones a Excel/CSV (ResultadosExport) y documentación técnica del módulo	Capas 1, 2 y 5 (fundamentos + Laravel Excel)

Nota: Esta división evita que una sola persona sea "la única que sabe WebSocket". Cada quien es responsable de una pieza, pero el código de las 4 piezas se integra en el mismo proyecto compartido, así que todos terminan leyendo y entendiendo el trabajo de los demás.

6.2 Día 1 (Lunes) — Cimientos, todos juntos

Objetivo del día: que los 4 tengan el entorno funcionando y el equipo entienda su rol.

- ☐ Todos: instalar PHP, Composer, Node.js y Git (Sección 2).
- ☐ Todos: clonar el repositorio compartido y correr composer install y npm install.
- ☐ Todos: copiar .env.example a .env, generar la APP_KEY y confirmar que php artisan serve levanta el proyecto.
- ☐ Integrante 1: revisa la estructura de carpetas del proyecto (app/Helpers, app/Imports, app/Exports) y confirma que los 4 la entienden.
- ☐ Todos: reunión corta (30 min) para acordar el "contrato de datos": ¿qué columnas exactas trae el Excel de estudiantes?, ¿qué eventos manda el WebSocket y con qué datos?, ¿cómo se llamará el archivo de resultados exportado?
- ☐ Integrante 1: agenda una reunión breve con un representante de Backend y Frontend para confirmar cómo se comunicarán los tres equipos — no tiene que resolverse hoy, pero sí quedar agendada.

6.3 Día 2 (Martes) — Aprendizaje dirigido

Mañana (todos juntos): repaso rápido y colaborativo de fundamentos de Laravel (Capa 2) — pueden hacerlo en modalidad "pair programming" en parejas, rotando.

Tarde (por rol):

- ☐ Integrante 1: termina de investigar Composer y deja el .env base documentado para que los demás solo hagan git pull.

- ☐ Integrante 2: investiga a fondo el protocolo WebSocket (Capa 6) y hace un primer boceto (aunque no funcione aún) de `websocket_server.php`.
- ☐ Integrante 3: investiga Laravel Excel — Import (Capa 5) y crea un Excel de prueba con 5-6 estudiantes de ejemplo.
- ☐ Integrante 4: investiga Laravel Excel — Export (Capa 5) y crea una exportación de prueba con datos inventados.

6.4 Día 3 (Miércoles) — Primer código funcional

- ☐ Integrante 2: escribe `NumeroALetras.php` y lo registra como directiva Blade en `AppServiceProvider`. Sube su avance al repositorio.
- ☐ Integrante 3: escribe `StudentsImport.php` (Sección 4.4), probándolo con el Excel de prueba.
- ☐ Integrante 4: escribe `ResultadosExport.php` (Sección 4.5) con datos de prueba inventados y confirma que el archivo se descarga bien.
- ☐ Integrante 1: revisa que el proyecto compile y corra con el código de los 3 compañeros integrado, ayudando a resolver cualquier error de Composer o de configuración.
- ☐ Todos: stand-up de 15 minutos al final del día — cada quien cuenta qué hizo y en qué se atascó.

6.5 Día 4 (Jueves) — Integración

- ☐ Integrante 2: programa una primera versión funcional de `websocket_server.php` (handshake + broadcast simple, aunque sea sin autenticación todavía) y de `WebSocketBroadcast.php`.
- ☐ Integrante 2 + Integrante 1 (pareja): agregan la autenticación por token al servidor WebSocket (Sección 4.3) y prueban con Postman o un cliente simple.
- ☐ Integrante 3: agrega el cifrado de votos con Crypt (Sección 4.7) y las validaciones compartidas (Sección 4.8).
- ☐ Integrante 4: mejora la exportación con formato (encabezados en negrita) y confirma que funciona con datos reales, no inventados.
- ☐ Integrante 1: conversa con Backend y Frontend según lo agendado el Día 1, y documenta el acuerdo final de integración (formato de eventos WebSocket, columnas del Excel).
- ☐ Todos: revisión cruzada de código (cada quien lee el módulo de un compañero y hace preguntas o sugerencias).

6.6 Día 5 (Viernes) — Prueba integral y demo

- ☐ Todos: prueba de extremo a extremo — importar estudiantes desde Excel, simular un voto, confirmar que el mensaje llega por WebSocket, y exportar resultados.
- ☐ Todos: prueba con un Excel de prueba más grande (20-30 filas), incluyendo filas intencionalmente dañadas (celdas vacías, datos mal escritos) para confirmar que la importación las detecta sin tumbar el programa.
- ☐ Todos: mini-demo interna del equipo Middleend: mostrar la importación, el aviso en tiempo real y la exportación funcionando juntas.
- ☐ Todos: retro rápida (¿qué costó más?, ¿qué aprendimos?) y armar el backlog de la Semana 2 (por ejemplo: manejo de archivos más grandes, pruebas automáticas más completas, documentación final).

Mensaje final: No necesitan llegar a la Semana 2 siendo expertos en Laravel o en WebSocket — necesitan llegar con un módulo que funciona, aunque sea sencillo, y con los 4 integrantes entendiendo cada pieza del rompecabezas. Avancen paso a paso, apóyense entre ustedes, y usen la Sección 5 cada vez que algo no tenga sentido. ¡Éxitos, equipo Middleend!

7. Glosario Express

Término	Significado breve
PHP	Lenguaje de programación en el que está escrito todo el proyecto
Laravel	Framework (conjunto de herramientas) de PHP que organiza el proyecto
Composer	Gestor que descarga librerías externas de PHP y organiza el proyecto
composer.json	Archivo de configuración de las dependencias del proyecto en PHP
Dependencia	Librería externa que el proyecto usa (ej. Laravel Excel)
Helper	Clase o función de apoyo reutilizable en cualquier parte del sistema
Service Provider	Lugar donde Laravel registra servicios y directivas especiales al arrancar
Directiva Blade	Comando corto y personalizado que se puede usar dentro de una vista (ej. @num2letras)
WebSocket	Tecnología que permite una conexión abierta y en dos vías entre el servidor y el navegador
Handshake	El "saludo" inicial que convierte una conexión normal en una conexión WebSocket
Broadcast	Enviar un mismo mensaje a todos los clientes conectados al mismo tiempo
Token de autenticación	Código secreto que confirma que quien se conecta tiene permiso de hacerlo
Laravel Excel	Paquete de Laravel para leer (Import) y escribir (Export) archivos de Excel
Import	Clase que define cómo se lee un archivo Excel y qué hacer con cada fila
Export	Clase que define cómo se genera un archivo Excel de salida
Cifrado (Crypt)	Proceso que "esconde" un dato para que no se pueda leer directamente
.env	Archivo con datos de configuración que cambian según la computadora donde corre el proyecto
Excepción	Un error que ocurre mientras el programa se ejecuta
PHPUnit	Librería para escribir pruebas automáticas en PHP, integrada en Laravel
Prueba unitaria	Código que verifica automáticamente que otro código hace lo que debe hacer
Definición de Terminado (DoD)	Lista de condiciones que debe cumplir una tarea para considerarse realmente lista